C Source Code Reading Exercise — Permutation Generator

Focus: code comprehension, overflow safety, output abstraction, file I/O correctness, and testing
Source curriculum: Permutation Generator in C, architected to safety-critical standards.
Learning Engine basis: Mastery-loop exercise: definitions → focused reading → MCQ with rationales → open questions → testing/homework.
Estimated time: 45–60 minutes plus optional coding extension.

Learning outcomes

By the end of this exercise, you should be able to:

- Explain what the program generates and how base-N counting maps integers to character sequences.
- Trace the flow from `main` through `generate_permutations` to the output sink.
- Justify the overflow guard in `safe_uint64_power` and identify edge cases.
- Describe why `OutputSink` improves testability and separation of concerns.
- Review file-writing code for correctness, failure handling, and resource cleanup.

How to use this exercise

Read the source excerpt in the appendix first. Then complete Sections A–E without compiling. Use Section F only after you have written your own answers.

Section A — Guided code survey

A1. In one sentence, state the program's observable output when `permutation_length` is `4`.

> Answer:

A2. Identify the five major sections of the program and write the responsibility of each section.

Section	Responsibility
1. Configuration and data definitions	
2. Abstract interface for output (`OutputSink`)	
3. Core logic / generator	
4. Concrete `FileSink` implementation	
5. System assembler (`main`)	

A3. What is the value of `ALPHABET_SIZE`? Show how you counted it.

> Answer:

A4. Why does the alphabet include both a space character and a newline character? What consequence does this have for the output file?

> Answer:

A5. What invariant should hold for `current_perm` immediately before each call to `sink->write`?

> Answer:

Section B — Trace the generator

For this section, pretend the alphabet is shortened to `{'a','b','c'}` and `length = 2`. This makes tracing manageable while preserving the algorithm.

`i`	`temp_row` evolution	`j = 1` char	`j = 0` char	generated string
0				
1				
2				
3				
4				
5				
6				
7				
8				

B1. Explain why the inner loop counts backward from `length - 1` to `0`.

> Answer:

B2. Describe the relationship between `i`, repeated division by `ALPHABET_SIZE`, and base-N representation.

> Answer:

B3. What would change if the inner loop counted upward from `0` to `length - 1`?

> Answer:

Section C — Multiple choice with justification

Choose one option for each question. Then write one sentence justifying why your option is correct and one sentence explaining the misconception behind one incorrect option.

C1

Why does `safe_uint64_power` test `*result > UINT64_MAX / base` before multiplication?

A. To avoid division by zero in the next loop iteration.
B. To detect whether multiplying by `base` would overflow `uint64_t`.
C. To ensure `exp` is less than `base`.
D. To round the result down to a safe value.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C2

What design role does `OutputSink` play?

A. It hides the alphabet from the generator.
B. It lets the generator write to any compatible destination without knowing whether it is a file, buffer, or test double.
C. It automatically retries failed writes.
D. It prevents all memory errors at compile time.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C3

What does `fwrite(buffer, sizeof(char), size, p) == size` check?

A. That exactly `size` characters were written.
B. That the file contains no newline characters.
C. That the output file was opened in binary mode.
D. That `buffer` is null-terminated.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C4

Which input validation is performed in `main` before generation?

A. It checks that the filename is non-empty.
B. It checks that `permutation_length` is nonzero and not greater than `MAX_PERMUTATION_LENGTH`.
C. It checks that the alphabet contains no duplicate characters.
D. It checks that disk space is sufficient for the output.

> Selected option:
> Justification:
> Misconception in one incorrect option:

C5

Which issue is most worth discussing in a safety-critical review of this code?

A. `current_perm` is not null-terminated, but the program writes by explicit length.
B. The generator relies on a huge output size, so runtime and disk exhaustion are possible even when integer overflow is prevented.
C. `uint64_t` cannot represent positive numbers.
D. `fputc` always fails after `fwrite`.

> Selected option:
> Justification:
> Misconception in one incorrect option:

Section D — Open response / construction tasks

D1. Overflow proof sketch. Write a short proof that the guard in `safe_uint64_power` prevents wraparound when `base > 0`. Include the precondition you need.

> Answer:

D2. Failure path analysis. List every place generation can return `false`. For each place, state what resource is open at that moment and who is responsible for cleanup.

> Answer:

D3. Test double design. Design an in-memory `OutputSink` that stores output into a buffer. Specify its context struct fields and two behaviours you would assert in tests.

> Answer:

D4. Property-based test. Propose a property-based test that would catch a broken overflow guard. Include generator inputs, expected invariant, and at least two edge cases.

> Answer:

D5. Safety-critical improvement. Recommend two changes that would make the program safer for production use. Explain the risk each change addresses.

> Answer:

Section E — Code review checklist

Item	Question	Evidence from code	Status
Bounds	Can all array writes to `current_perm` be shown within `0..MAX_PERMUTATION_LENGTH-1`?		
Overflow	Is every arithmetic growth step guarded before it can overflow?		
I/O	Are partial writes and write failures detected?		

Item	Question	Evidence from code	Status
Cleanup	Is the file closed on both success and generation failure?		
Testability	Can generator logic be tested without writing a real file?		
Scalability	Can the program avoid infeasible output volumes?		

Section F — Answer key and marking guide

Use this section after attempting the exercise. Award partial credit for precise reasoning even where wording differs.

MCQ key

Question	Answer	Core rationale
C1	B	The comparison checks whether `result × base` would exceed `UINT64_MAX` before multiplication happens.
C2	B	The sink interface decouples generation from the output destination and enables test doubles.
C3	A	`fwrite` returns the number of elements written; here each element is one `char`.
C4	B	`main` rejects zero length and values above `MAX_PERMUTATION_LENGTH`.
C5	B	Overflow safety does not guarantee feasible runtime, output size, or disk capacity.

Open-response rubric

Task	4 points	2 points	0–1 point
D1	States precondition `base > 0` and explains division guard before multiplication.	Mentions overflow guard but proof is incomplete.	Only restates code or misses overflow mechanism.
D2	Finds overflow, write, and `write_char` failures; notes `main` closes file after generation.	Finds some failure paths or cleanup partly.	Misses most error paths.
D3	Defines context buffer/capacity/length and useful assertions.	Gives a plausible sink with limited assertions.	Does not preserve interface contract.
D4	Uses edge cases near `UINT64_MAX` and checks false/no wraparound.	Has tests but weak edge cases.	Only tests ordinary small values.
D5	Links changes to risks such as disk exhaustion, null sink, base zero, filename/config validation.	Suggests changes with vague risk explanation.	Cosmetic-only changes.

Spaced retrieval prompts

- Tomorrow: Explain the overflow guard without looking at the code.
- In three days: Re-trace the shortened alphabet example for `length = 3`.
- In one week: Write a memory-backed `OutputSink` from scratch.
- In two weeks: Review the program for infeasible output size and propose a mitigation.

Appendix — Source excerpt for reading

Read this source carefully before completing the exercise. Line numbers are not included, so refer to function names and code fragments in your answers.

```c
/**
 * @file main.c
 * @brief Permutation Generator in C, architected to safety-critical standards.
 *
 * @author Dominic Alexander Cooper (Original Algorithm)
 * @author Gemini (Architectural Refactoring)
 *
 * @details
 * This program generates all possible character combinations for a given length,
 * based on a predefined character set. It is a complete rewrite of the original
 * concept to align with principles of safety-critical software design.
 */
#include <stdio.h>
#include <stdint.h>  // For fixed-width integer types like uint64_t
#include <stdbool.h> // For bool type
#include <limits.h>  // For UINT64_MAX

//============================================================================
// 1. CONFIGURATION AND DATA DEFINITIONS
//============================================================================
#define MAX_PERMUTATION_LENGTH 10

static const char ALPHABET[] = {
    'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k', 'l', 'm',
    'n', 'o', 'p', 'q', 'r', 's', 't', 'u', 'v', 'w', 'x', 'y', 'z',
    'A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J', 'K', 'L', 'M',
    'N', 'O', 'P', 'Q', 'R', 'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z',
    '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', ' ', '\n'
};

static const uint64_t ALPHABET_SIZE = sizeof(ALPHABET) / sizeof(ALPHABET[0]);

//============================================================================
// 2. ABSTRACT INTERFACE FOR OUTPUT (OutputSink)
//============================================================================
typedef struct {
    void* context;
    bool (*write)(void* context, const char* buffer, uint64_t size);
    bool (*write_char)(void* context, char c);
} OutputSink;
```

```cpp
//============================================================================
// 3. CORE LOGIC (Generator)
//============================================================================
static bool safe_uint64_power(uint64_t base, uint64_t exp, uint64_t* result) {
    *result = 1;

    for (uint64_t i = 0; i < exp; ++i) {
        if (*result > UINT64_MAX / base) {
            return false;
        }

        *result *= base;
    }

    return true;
}

static bool generate_permutations(uint64_t length, OutputSink* sink) {
    uint64_t num_combinations;

    if (!safe_uint64_power(ALPHABET_SIZE, length, &num_combinations)) {
        return false;
    }

    char current_perm[MAX_PERMUTATION_LENGTH];

    for (uint64_t i = 0; i < num_combinations; ++i) {
        uint64_t temp_row = i;

        for (int64_t j = length - 1; j >= 0; --j) {
            uint64_t char_index = temp_row % ALPHABET_SIZE;
            current_perm[j] = ALPHABET[char_index];
            temp_row /= ALPHABET_SIZE;
        }

        if (!sink->write(sink->context, current_perm, length)) {
            return false;
        }

        if (!sink->write_char(sink->context, '\n')) {
            return false;
        }
    }

    return true;
}

//============================================================================
// 4. CONCRETE IMPLEMENTATION OF OUTPUTSINK (FileSink)
//============================================================================
static bool file_sink_write(void* context, const char* buffer, uint64_t size) {
    FILE* p = (FILE*)context;
    return fwrite(buffer, sizeof(char), size, p) == size;
}
```

```c
static bool file_sink_write_char(void* context, char c) {
    FILE* p = (FILE*)context;
    return fputc(c, p) != EOF;
}

static bool file_sink_init(OutputSink* sink, const char* filename) {
    FILE* p = fopen(filename, "w");

    if (p == NULL) {
        return false;
    }

    *sink = (OutputSink){
        .context = p,
        .write = file_sink_write,
        .write_char = file_sink_write_char
    };

    return true;
}

static void file_sink_close(OutputSink* sink) {
    if (sink && sink->context) {
        fclose((FILE*)sink->context);
        sink->context = NULL;
    }
}

//============================================================================
// 5. SYSTEM ASSEMBLER (main)
//============================================================================
int main(void) {
    const uint64_t permutation_length = 4;

    if (permutation_length == 0 || permutation_length > MAX_PERMUTATION_LENGTH) {
        return 1;
    }

    OutputSink file_sink;

    if (!file_sink_init(&file_sink, "system_safe.txt")) {
        perror("Error opening file");
        return 1;
    }

    bool success = generate_permutations(permutation_length, &file_sink);

    file_sink_close(&file_sink);

    if (!success) {
        return 1;
    }
```

```
    return 0;
}
```